UNITREE R1 · MACBOOK PRO M3 · PROJECT REPORT

Seeing, scanning and moving a humanoid

Six workstreams over the project so far. Video, VR and 3D capture are working end to end. Motion is built and blocked — not by our code, but by the robot refusing external commands while its own AI mode holds the joints.

---

32 Python modules ~6,400 lines 370-frame scan captured Report compiled 3 Aug 2026

● WORKING

Video pipeline

RTSP off the robot, re-encoded to MJPEG, 150–300 ms glass to glass.

● WORKING

DDS networking

Unicast peer discovery patched in where Unitree's SDK only does multicast.

● WORKING

VR headset

Self-contained WebXR page over TLS. No app, no sideloading, no Unity.

● WORKING

3D scan capture

370 sharp frames, fisheye corrected to pinhole, ready for reconstruction.

● BLOCKED

Arm gestures

All 22 wired up. Robot returns `7406` — worked earlier the same day.

● BUILT, UNVERIFIED

Walking

Two backends and full interlocks. `SetVelocity` refused with `127`.

Video pipeline

● WORKING

RTSP → MJPEG → HEADSET

The robot exposes H.264 over RTSP at `rtsp://192.168.123.161:8551/video`. The Quest browser cannot play RTSP, so the Mac pulls the stream, re-encodes each frame as JPEG and serves it as multipart MJPEG — which every browser plays natively.

MJPEG was chosen against two alternatives. HLS buffers several seconds, which is unusable for looking around through a robot. WebRTC would be lower latency still but needs a signalling server and TURN setup. MJPEG lands at 150–300 ms on a local network in about forty lines.

Frame sources are pluggable — DDS, RTSP, HTTP and local webcam all expose the same three calls — because Unitree does not document one blessed video path across their humanoids.

DDS networking

● WORKING

THE HARDEST THING THAT NOW WORKS QUIETLY

Unitree's `ChannelFactoryInitialize` hardcodes a CycloneDDS config containing only an interface name — no `<Peers>` — and passes it straight to `Domain()`, overriding `CYCLONEDDS_URI`. Discovery is therefore multicast only.

That is fine on the flat `192.168.123.x` network Unitree assume, and fails across a routed hop. Patching the config template in before initialisation adds unicast peer discovery, which does cross one. Port sweeping established that only `:7400` is bound, so the peer list targets it explicitly.

> ADDED THIS SESSION
>
> The USB-ethernet adapter does not keep a stable name — it has appeared as `en5`, `en6` and `en7` across replugs, and every tool hardcoding `en6` failed with `channel factory init error`, which says nothing about why. Interfaces are now found by the robot's subnet instead of by name, and a missing adapter reports itself in a sentence.

VR headset interface

● WORKING

WEBXR · WEBGL2 · NO DEPENDENCIES

A single self-contained page, hand-rolled WebGL2 with no Three.js or A-Frame, so there is nothing to fetch and nothing to sideload. WebXR supplies the per-eye projection and view matrices, so the page builds a mesh and hands those matrices to the shader.

It is served over TLS with a self-signed certificate, because `navigator.xr` is undefined on insecure origins — over plain HTTP the Quest reports "WebXR not available" regardless of what it supports. Port 443 keeps the headset URL down to `https://<ip>` with no port to type.

Screen shape, rotation, size and distance are all adjustable from inside the headset, because guessing the right orientation from outside and restarting the server each time is miserable.

PROBLEM	RESOLUTION
`navigator.xr undefined`	self-signed TLS to reach a secure context
`sensor mounted sideways`	texture-coordinate rotation, adjustable live
`BrokenPipeError spam`	server swallows headset keep-alive drops
`only 4 of 22 gestures reachable`	buttons generated from the robot's own action list

3D scan capture

● WORKING

SLAM3R · MONOCULAR RGB · ARXIV 2412.09401

SLAM3R reconstructs a dense point cloud from monocular RGB video alone — no depth, stereo, LiDAR or IMU. That matches the R1's single camera exactly. It requires CUDA, so the work splits: the Mac captures, a GPU machine reconstructs.

The capture tool is not a recorder. It paces frames at a steady 6 fps, rejects motion-blurred frames against a threshold that self-calibrates from the scene, and shows live overlap between consecutive frames — the single most useful signal during a scan, since frames that do not overlap cannot be registered to each other.

SCAN	MEASURED
frames kept	370 of 1,268 seen — 129 rejected for blur
duration	83 s at 6 fps
sharpness	median 502, worst 308
overlap	median 0.87, 10th pct 0.66
output	370 pinhole frames, 720 × 603, 53 MB

THE BLOCKER THAT HAD TO BE SOLVED FIRST

The R1's camera is a hard fisheye, and SLAM3R's own documentation lists wide-angle lenses and unfamiliar distortion as known failure modes — it inherits DUSt3R's assumption of roughly pinhole imagery. Feeding the raw frames in was the most likely way to get a warped reconstruction.

Proper calibration needs the robot in front of a printed chessboard, which was not available. Instead the lens field of view was **measured from the scene itself**: undistort at each candidate FOV, run a line-segment detector, and keep the value that maximises total detected straight-line length. A factory cell is full of genuinely straight aluminium extrusion, so the correct correction is the one that makes those lines longest. The sweep found a clean peak at **160°**.

Output is square rather than landscape, because DUSt3R's loader resizes the short side to 224 and centre-crops — a 1280 × 720 frame would have lost 44% of its width before the network ever saw it, and the discarded pixels happen to be the worst-distorted edges anyway.

Arm gestures

● BLOCKED BY THE ROBOT

22 ACTIONS WIRED · EXECUTION REFUSED

The robot's own action list was read back over DDS and all 22 gestures are now exposed in the headset page, generated from that list rather than hardcoded. Four were previously reachable; the other eighteen required editing the server.

They worked earlier in the day and do not now. Every gesture returns `7406` — including `release_arm`, the null one that simply lets the arms down. Reads still succeed, so the service is reachable and healthy; only execution is refused.

HEALTH CHECK	RESULT	
GetActionList	success — 22 actions	● OK
ExecuteAction(23) right_hand_up	7406	● REFUSED
ExecuteAction(99) release_arm	7406	● REFUSED
arm joints 15–19, 22–26	mode 1, holding 45°, 35–44 °C	● HEALTHY
motor fault flags	none — all motorstate 0	● HEALTHY
posture	upright, roll/pitch ≈ 1°	● HEALTHY

The joints reading `mode 0` all report `temp 0` — they are absent hardware, not disabled. The R1 has 5-DOF arms, not 7.

Walking

● BUILT, UNVERIFIED

TWO BACKENDS · THREE INTERLOCKS

Thumbstick driving is implemented end to end and tested against a dry-run harness — clamping, deadzone, watchdog timing, send rate, HTTP endpoints and shutdown all verified. It is off unless explicitly enabled, because starting the video feed should never be able to make a humanoid take a step.

Stopping is layered deliberately. The stick is spring-centred, so letting go *is* a zero command — the same reason Unitree's own controller needs no hold-to-run button. Behind that sits a watchdog, and behind that a per-command expiry that runs on the robot itself, so a crashed Mac or a pulled cable stops it with none of our code involved. Zero velocity keeps the robot standing and balancing; nothing on a stick or button can make it go limp.

What it cannot yet do is walk. `SetVelocity` returns `127` — refused, not missing, since a missing call would return `3203` — and the robot's state machine is unchanged by the attempt. A second backend was written that publishes to `rt/wirelesscontroller`, the gamepad channel the handheld remote drives and the one the robot's AI mode actually listens to. It is untested against hardware.

> CORRECTION WORTH RECORDING
>
> An early diagnosis blamed the service name: the SDK contradicts itself, with `g1/loco` hardcoded to `"sport"` while `h1/loco` uses `"loco"`, and this robot advertises command topics for both. Probing disproved it — `loco` is not reachable at all and `sport` was correct from the start. The probe was written precisely because guessing here is silent.

The one root cause

● OPEN

MEASURED, READ-ONLY, AGAINST THE ROBOT

Two services refusing, two different error codes, one explanation. The motion switcher reports that a mode named `ai` owns the joints, and while it does, external motion commands are rejected while reads continue to succeed.

PROBE	RESULT	READING
sport · GetFsmId	ok, FSM 811	service alive
sport · getters 7002–7005	API not implemented	1 of 5 calls exists
sport · SetVelocity(0,0,0)	127	exists, refused
loco service	not reachable	wrong path
motion_switcher · CheckMode	{form 0, name 'ai'}	**owns the joints**
motion_switcher · SelectMode('normal')	7004	name not valid here
arm · ExecuteAction	7406	same refusal shape
rt/lowstate	live, ~10 Hz	link healthy
rt/wirelesscontroller	silent	remote not bridged

The decisive detail is that the gestures worked earlier in the same session with no code change in between. The robot's configuration moved, not ours. Resolving it is an app or mode question rather than a programming one.

Separately, the USB-ethernet adapter has been physically dropping. It disappeared entirely mid-session and the robot stopped responding to ping. That accounts for the intermittent DDS initialisation failures and the RTSP stream timeouts throughout, and it needs fixing before any motion testing is trustworthy.

Diagnostics built along the way

ALL READ-ONLY UNLESS STATED

TOOL	ANSWERS
`loco_probe.py`	which locomotion service answers, the FSM id, and which mode owns the joints
`robot_state.py`	live lowstate — per-joint mode, angle, torque, temperature, posture, remote decode
`wireless_probe.py`	whether the handheld remote reaches DDS at all
`undistort.py`	fisheye → pinhole, with FOV measured from scene line straightness
`r1_walk.py`	keyboard driving, so first motion happens where you can see the robot
`control_probe · deep_probe · field_probe`	earlier DDS, topic and network discovery that made the rest possible

What comes next

IN DEPENDENCY ORDER

— Reseat the ethernet adapter. Nothing below is trustworthy while the link flaps.

— Settle the joint-ownership question in the Unitree app — whether an app session holds control, and which mode permits external commands. This unblocks gestures and walking together.

— Run SLAM3R on the undistorted frames. They are prepared and packaged; this needs a CUDA machine and nothing else.

— Prove `rt/arm_sdk` can move one joint 10°. It is a blended path — arm commands mix in while the robot keeps balancing on its own legs — and it is separate from the refusing gesture service, so it may work regardless.

— Only then, teleoperation. The arm is 5-DOF, giving wrist position, elbow swivel and forearm roll but not hand pitch or yaw — which matters for aiming a gripper, and this robot has none.